

C++ PROGRAMMING

L A B 1

Enumeration . Function Overloading . Default Arguments . Class & Objects

SUBJECT
C++ Programming

LAB
Laboratory 1

QUESTIONS
7 Programs

Q1 WAP Using Constant Qualifier**CONCEPT**

The const keyword makes a variable read-only after initialization. It is stored in protected memory and any attempt to modify it triggers a compile-time error, preventing accidental value changes.

>> PROGRAM CODE

```
#include <iostream>
using namespace std;

int main() {
    const float PI      = 3.14159f;
    const int  MAX_SIZE = 100;
    float radius;

    cout << "Enter radius: ";
    cin  >> radius;

    float area = PI * radius * radius;
    cout << "Area of Circle = " << area      << endl;
    cout << "Max Size       = " << MAX_SIZE << endl;

    // PI = 3.14;  // COMPILE ERROR
    return 0;
}
```

>> SAMPLE OUTPUT

```
Enter radius: 7
Area of Circle = 153.938
Max Size       = 100
```

*** KEY POINTS**

- > const variables must be initialized at the point of declaration.
- > Stored in read-only memory -- modification causes a compile-time error.
- > Improves code safety and communicates intent clearly to other developers.

Q2 WAP to Define Enumerator with Custom Integer Values**CONCEPT**

!

An enum (enumeration) is a user-defined type consisting of named integer constants. By default values start from 0; assigning explicit values improves readability and control over underlying integer representation.

>> **PROGRAM CODE**

```
#include <iostream>
using namespace std;

enum Planet {
    MERCURY = 1,
    VENUS   = 2,
    EARTH   = 3,
    MARS    = 4,
    JUPITER = 10,
    SATURN  = 11
};

int main() {
    Planet home = EARTH;
    cout << "MERCURY value : " << MERCURY << endl;
    cout << "VENUS   value : " << VENUS   << endl;
    cout << "EARTH   value : " << EARTH   << endl;
    cout << "MARS    value : " << MARS    << endl;
    cout << "JUPITER value : " << JUPITER << endl;
    cout << "SATURN  value : " << SATURN  << endl;
    cout << "Home planet : " << home    << endl;
    return 0;
}
```

>> **SAMPLE OUTPUT**

```
MERCURY value : 1
VENUS   value : 2
EARTH   value : 3
MARS    value : 4
JUPITER value : 10
SATURN  value : 11
Home planet : 3
```

*** KEY POINTS**

- > Enumerators without explicit values auto-increment from the previous value.
- > Enum constants improve readability by replacing magic numbers with descriptive names.
- > The underlying type is int; casting to int returns the assigned integer value.

Q3 WAP to Find Area Using Function Overloading (Without Class)**CONCEPT**

Function overloading defines multiple functions sharing the same name but different parameter signatures. The compiler resolves which version to call at compile-time based on argument types and count -- this is static (compile-time) polymorphism.

>> PROGRAM CODE

```
#include <iostream>
using namespace std;

// Overload 1: Square (1 argument)
float area(float side) {
    return side * side;
}

// Overload 2: Rectangle (2 arguments)
float area(float length, float breadth) {
    return length * breadth;
}

// Overload 3: Triangle
float area(float base, float height, int) {
    return 0.5f * base * height;
}

int main() {
    cout << "Square    (side=5)      : " << area(5.0f)          << endl;
    cout << "Rectangle (6 x 4)      : " << area(6.0f, 4.0f)       << endl;
    cout << "Triangle  (base=8, h=5) : " << area(8.0f, 5.0f, 0) << endl;
    return 0;
}
```

>> SAMPLE OUTPUT

```
Square    (side=5)      : 25
Rectangle (6 x 4)      : 24
Triangle  (base=8, h=5) : 20
```

*** KEY POINTS**

- > Functions are distinguished by their signature (name + parameter types/count), NOT by return type.
- > The dummy int parameter in area() for triangle distinguishes it from the rectangle version.
- > Overloading resolution happens entirely at compile time -- zero runtime overhead.

Q4 Repeat Program 3 Using Class and Objects**CONCEPT**

!

A class bundles related data and behaviour together. Overloaded member functions inside a class are invoked through an object using the dot (.) operator, combining OOP encapsulation with compile-time polymorphism.

>> **PROGRAM CODE**

```
#include <iostream>
using namespace std;

class Shape {
public:
    float area(float side) {
        return side * side;
    }
    float area(float length, float breadth) {
        return length * breadth;
    }
    float area(float base, float height, int) {
        return 0.5f * base * height;
    }
};

int main() {
    Shape s;
    cout << "Square      : " << s.area(5.0f) << endl;
    cout << "Rectangle  : " << s.area(6.0f, 4.0f) << endl;
    cout << "Triangle   : " << s.area(8.0f, 5.0f, 0) << endl;
    return 0;
}
```

>> **SAMPLE OUTPUT**

```
Square      : 25
Rectangle   : 24
Triangle    : 20
```

*** KEY POINTS**

- > The dot operator (.) is used to invoke member functions on an object.
- > Multiple objects of the same class can exist independently in memory.
- > Wrapping logic in a class improves reusability, modularity, and maintainability.

Q5 WAP to Add Three Integers Using Default Arguments**CONCEPT**

Default arguments supply fallback values for function parameters when the caller omits them. They must always appear at the trailing end of the parameter list and are specified only in the function declaration.

>> PROGRAM CODE

```
#include <iostream>
using namespace std;

int addThree(int a = 10, int b = 20, int c = 30) {
    cout << "   Called with: a=" << a << ", b=" << b << ", c=" << c << endl;
    return a + b + c;
}

int main() {
    cout << "All defaults      -> Sum = " << addThree()          << endl;
    cout << "One arg (5)      -> Sum = " << addThree(5)          << endl;
    cout << "Two args (5,15)   -> Sum = " << addThree(5, 15)       << endl;
    cout << "All explicit     -> Sum = " << addThree(5, 15, 25)    << endl;
    return 0;
}
```

>> SAMPLE OUTPUT

```
All defaults      ->   Called with: a=10, b=20, c=30      Sum = 60
One arg (5)       ->   Called with: a=5,  b=20, c=30      Sum = 55
Two args (5,15)   ->   Called with: a=5,  b=15, c=30      Sum = 50
All explicit      ->   Called with: a=5,  b=15, c=25      Sum = 45
```

*** KEY POINTS**

- > Default arguments must be trailing -- you cannot skip a middle parameter.
- > They reduce the need for multiple overloaded versions of the same function.
- > If a separate declaration exists, defaults go in the declaration, NOT the definition.

Q6**WAP Using Class & Object -- Member Function to Initialize Data Members****CONCEPT****!**

Data members declared private are accessible only through public member functions (setters/getters). This encapsulation principle protects object state and ensures data is always modified through a controlled interface.

>> PROGRAM CODE

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int    rollNo;
    float  marks;

public:
    void setData(string n, int r, float m) {
        name  = n;
        rollNo = r;
        marks  = m;
    }
    void display() {
        cout << " Name      : " << name << endl;
        cout << " Roll No  : " << rollNo << endl;
        cout << " Marks    : " << marks << "%" << endl;
    }
};

int main() {
    Student s1, s2;
    s1.setData("Alice", 101, 94.5f);
    s2.setData("Bob", 102, 87.0f);
    cout << "--- Student 1 ---" << endl;
    s1.display();
    cout << "--- Student 2 ---" << endl;
    s2.display();
    return 0;
}
```

>> SAMPLE OUTPUT

```
--- Student 1 ---
Name      : Alice
Roll No   : 101
Marks     : 94.5%
--- Student 2 ---
Name      : Bob
Roll No   : 102
Marks     : 87%
```

*** KEY POINTS**

- > private data members enforce data hiding -- external code cannot access them directly.
- > Each object (s1, s2) maintains its own independent copy of name, rollNo, and marks.
- > Setter/getter pattern is the foundation of encapsulation in OOP.

Q7 Create Class Customer -- Discount 5% if Amount > 20,000**CONCEPT**

!

A class models a real-world entity (Customer) with attributes (name, product, amount) and behaviour (takeInfo, displayInfo). Business logic (discount calculation) is encapsulated inside the class, keeping main() clean and simple.

>> **PROGRAM CODE**

```
#include <iostream>
#include <string>
using namespace std;

class Customer {
private:
    string customerName;
    string productBought;
    float paidAmount;

public:
    void takeInfo() {
        cout << "Enter Customer Name : ";
        cin.ignore();
        getline(cin, customerName);
        cout << "Enter Product Bought : ";
        getline(cin, productBought);
        cout << "Enter Amount Paid : Rs. ";
        cin >> paidAmount;
    }
    void displayInfo() {
        float discount = 0.0f;
        float finalAmount = paidAmount;
        if (paidAmount > 20000) {
            discount = paidAmount * 0.05f;
            finalAmount = paidAmount - discount;
        }
        cout << "\n=== BILL SUMMARY ===" << endl;
        cout << "Customer : " << customerName << endl;
        cout << "Product : " << productBought << endl;
        cout << "Amount : Rs. " << paidAmount << endl;
        if (discount > 0)
            cout << "Discount : Rs. " << discount << " (5%)" << endl;
        else
            cout << "Discount : No discount applied" << endl;
        cout << "TOTAL DUE : Rs. " << finalAmount << endl;
        cout << "======" << endl;
    }
};

int main() {
    Customer c;
    c.takeInfo();
    c.displayInfo();
    return 0;
}
```

>> **SAMPLE OUTPUT**

Amount > 20,000

```
Enter Customer Name : Ramesh
Enter Product Bought : Laptop
Enter Amount Paid   : Rs. 60000
```

```
=== BILL SUMMARY ===
```

```
Customer : Ramesh
Product  : Laptop
Amount   : Rs. 60000
Discount : Rs. 3000 (5%)
TOTAL DUE : Rs. 57000
=====
```

Amount <= 20,000

```
Enter Customer Name : Sita
Enter Product Bought : Book
Enter Amount Paid    : Rs. 500
```

```
=== BILL SUMMARY ===
```

```
Customer : Sita
Product  : Book
Amount   : Rs. 500
Discount : No discount applied
TOTAL DUE : Rs. 500
=====
```

*** KEY POINTS**

- > The discount condition (amount > 20000 -> 5%) is encapsulated inside displayInfo().
- > cin.ignore() clears the newline from the buffer before getline() to avoid skipping input.
- > Separation of takeInfo() and displayInfo() follows the Single Responsibility Principle.